

Number Guessing Game Final Report

Nicholas Piscopo CMPSC132

Project Description:

This project is a terminal based Number Guessing game that is implemented from Python. The program generates a random number within the selected range where the user gets to guess the number based on feedback provided after each guess feedback. The game has multiple difficulty levels and a try again feature, allowing for multiple rounds without restarting the program.

At the start of each round, the user selects between Easy (1-50), Medium (1-100), or Hard (1-200). Based on what is chosen determines the range of the random number. Then the user enters guesses where they are given feedback whether the guess is too high or too low. Once the user guesses correctly, the program displays a “Correct”! Congratulations!” followed by how many guesses it took for the user. Where they are then prompted with whether they want to go again or exit the program.

Approach and Logic Used:

The main logic of my program is built around loops and conditional statements. The loop is used to continuously push the user for guesses, while the conditional statements compare each guess to the randomly generated number and provide feedback. The game continues running until the correct number is guessed.

For my program I mainly used 4 functions:

- `select_level()`
 - Prompts the user to select the difficulty level
 - Uses conditional statements to assign a number range based on user's choice
 - Accepts both (1,2,3) and text input (easy, medium ,hard)
 - Defaults to medium if the input is invalid
- `read_guess()`

- Prompts the user to enter a guess in the selected range
- Uses a while loop to repeatedly request input until a valid value is entered
- Uses a try/except block to ensure the input is an integer
- `run_round()`
 - Display welcome message for the game
 - Calls `select_level()` to determine the number range
 - Uses a loop to repeatedly ask the user for guesses
 - Compares the user's guess to the target number using the if/Elif/else statements
 - Tracks the number of attempts made by the user
 - Ends the loop when the correct number is guesses and displays results
- `main()`
 - Controls the replay of the game
 - Uses a loop to allow multiple rounds
 - Asks the user to decide whether they want to continue playing
 - Ends the program when the user chooses not to

Data Structures:

The program primarily uses:

- **Integers** for sorting the random number, user's guesses, and attempt number
- **Strings** for user input of difficulty and displaying messages and feedback to the user

The use of functions helps organize logic to improve readability and to make it easier to debug.

Complex structures like lists or dictionaries were not necessary.

Challenges:

One of the main challenges I faced was handling invalid user input. Since all input is initially read as a string, I needed to ensure that the program could safely convert it into an integer without crashing. To solve this, I used a try/except block inside the `read_guess()` function to catch errors and prompt the user again if the input was not a valid number.

Another challenge was making sure the user's guess stayed within the correct range based on the selected difficulty. This required additional checks to ensure that the input value was within the allowed bounds before continuing the game.

Designing the loop structure was also an important part of the project. The program needed to continuously prompt the user until the correct number was guessed, while also tracking attempts and providing feedback after each guess. In addition, implementing the replay feature requires careful control of program flow so that the game can restart smoothly without restarting the entire program.

Improvements:

Some improvements that could be made to this project would be:

- Attempt Limit (to increase difficulty)
- Scoring System (This would make the game more rewarding for having less guesses)
- GUI (Having a visual application such as Tkinter would make it more interactive and user friendly)